



SECURITY ASSESSMENT LOOP Protocol Partial source review

September 18, 2026

Partial Source Review - score reflects reviewed scope only

BLADE POOL

RISK ANALYSIS | LOOP Protocol.

■ Classifications of Manual Risk Results

Note: Risk classifications follow CVSS v4.0 standards (<https://www.first.org/cvss/v4.0/specification-document>). Contract security metrics are verified through GoPlus API token scanner technology, which provides real-time validation of contract parameters including

Classification	CVSS Range	Description
 Critical	9.0 - 10.0	Critical vulnerabilities that pose immediate and severe risks requiring urgent attention.
 High	7.0 - 8.9	High-priority issues that could lead to significant security breaches or financial loss.
 Medium	4.0 - 6.9	Medium-severity findings that should be addressed to improve contract security.
 Low	0.1 - 3.9	Low-risk items or best practice suggestions with minimal security impact.
 Informational	0.0	Informational findings about detected functions or contract features.

■ Manual Code Review Risk Results

Contract Security	Description
 Buy Tax	N/A
 Sale Tax	N/A
 Cannot Buy	N/A
 Cannot Sale	N/A
 Max Tax	N/A
 Modify Tax	N/A
 Fee Check	Not Applicable
 Is Honeypot?	Not Applicable
 Trading Cooldown	Not Applicable
 Enable Trade?	N/A

Contract Security	Description
● Pause Transfer?	N/A
● Max Tx?	N/A
● Is Anti Whale?	N/A
● Is Anti Bot?	N/A
● Is Blacklist?	N/A
● Blacklist Check	N/A
● Is Whitelist?	N/A
● Can Mint?	Out of scope (handler)
● Is Proxy?	N/A
● Can Take Ownership?	Upgrade authority not verified on-chain
● Hidden Owner?	N/A
● Owner	Program upgrade authority (verify on-chain)
● Self Destruct?	N/A
● External Call?	Raydium CPMM CPI (raydium.rs - program id pinned)
● Other?	Solana Anchor staking + referral program. Reviewed modules: router + state/constants/utis/raydium/error. Handler layer excluded.
● Holders	N/A
● Audit Confidence	Partial - reviewed scope only (handler/account-validation layer excluded)
● Authority Check	Not reviewed (handler layer out of scope)
● Freeze Check	N/A

Overview of identified vulnerabilities and risks. See the full report for detailed methodology and recommendations.

CFG Ninja Verified on September 18, 2026



LOOP Protocol

Executive Summary

TYPES

DeFi

ECOSYSTEM

Solana

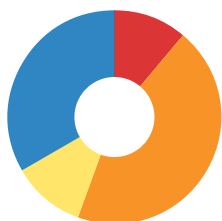
LANGUAGE

Rust

Timeline



Vulnerability Summary



9
Total Findings

0
Resolved

9
Pending

9
Unresolved

Partial source review: the provided modules (lib.rs router + state.rs, constants.rs, utils.rs, raydium.rs, error.rs) were manually reviewed and scored. The instruction-handler / account-validation layer (instructions/*.rs) was NOT provided and is EXCLUDED from this score; no whole-program pass/fail is issued. 6 security findings plus 3 informational in the reviewed scope.

Severity	Resolution Status	Description
0 Critical		Immediate danger. Can lead to loss of user funds, unauthorized access, or full system compromise.
1 High	0 Resolved, 1 Pending	Significant risk that should be addressed urgently to prevent exploitation or financial loss.
4 Medium	0 Resolved, 4 Pending	Moderate risk that can lead to issues if combined with other weaknesses; should be addressed.
1 Low	0 Resolved, 1 Pending	Minor concern or best-practice deviation with limited direct security impact.
3 Informational	0 Resolved, 3 Pending	Not a vulnerability - code-quality, documentation, or optimization suggestions.

PROJECT OVERVIEW | LOOP Protocol.

Token Summary

Parameter	Result
Address	81EUhaSVy8edRdyZ2TSrZSYa72U2tKY8zTDURsUAtRT6
Name	LOOP Protocol
Token Tracker	LOOP Protocol (LOOP)
Decimals	N/A
Supply	N/A (program)
Platform	Solana
Compiler	Anchor (Rust / Solana BPF)
Contract Name	staking (program)
Optimization	No
LicenseType	See source header
Language	Rust
Codebase	https://x.com/loop_protocol_

TECHNICAL FINDINGS

Smart contract security audits classify risks into several categories: Critical, High, Medium, Low, and Informational. These classifications help assess the severity and potential impact of vulnerabilities found in smart contracts.

Classification of Risk

Severity	CVSS Range	Description
 Critical	9.0 - 10.0	Immediate danger. Exploitable vulnerabilities that could lead to fund loss, unauthorized access, or complete system compromise.
 High	7.0 - 8.9	Significant security risk. Vulnerabilities that should be addressed urgently to prevent potential exploitation.
 Medium	4.0 - 6.9	Moderate security risk. Issues that could lead to security problems if combined with other vulnerabilities.
 Low	0.1 - 3.9	Minor security concern. Best practice violations or low-impact issues.
 Informational	0.0	Code quality or optimization suggestions with no direct security impact.

Detailed Security Findings

Each finding lists its severity, an indicative CVSS, the source location, a description, our recommendation and its current mitigation status. Findings apply to the reviewed modules only; where a finding's final impact depends on the excluded instruction-handler layer, this is stated in the finding.

LOOP-01 | Broad Privileged Control Over Prices, Fund Routing and Reward Parameters.

Category	Severity	CVSS	Location	Status
Centralization	 High	7.2	src/*.rs: lib.rs (router) / state.rs: StakeConfig	 Attention Required

Description

The design concentrates far-reaching economic control in the admin and vault_admin roles. From the router and StakeConfig alone, privileged instructions can set the A8/A9 prices used in economic conversions (set_prices -> a9_price/a8_price), redirect where protocol value flows (set_project_wallet, set_vault_admin), change staking economics (set_min_stake, set_pools, set_tier_daily_limit, set_seconds_per_day) and switch reward accrual on or off (set_rewards_enabled). An owner-set price is a direct value lever over what participants pay and receive; set_vault_admin / set_project_wallet control fee and fund destinations. NOTE: the authorization that gates these instructions lives in the instruction-handler modules, which were not provided and were not reviewed - so it is not confirmed that only the legitimate admin can invoke them. The powers themselves are inherent to the state/router design regardless of that check.

Recommendation

Minimise the trusted surface: move privileged parameters (prices, wallets, limits, seconds_per_day) behind a timelock and/or multisig, emit events on every privileged change, and publish the admin/vault_admin/upgrade-authority keys so users can verify them on-chain. Confirm in the handler layer that each set_* / operator / fund instruction enforces a strict admin (or operator) signer + has_one(config) constraint.

Mitigation

Open - centralization is a design property of the current state model; handler-level authorization was not reviewed (out of scope).

References:

N/A

LOOP-02 | High-Yield Multi-Level Referral Model Depends on Continuous Vault Funding.

Category	Severity	CVSS	Location	Status
Economic Design	 Medium	5.0	src/*.rs: constants.rs / utils.rs: settle_order_static, set tle_dynamic_rewards	 Attention Required

Description

The reward model is a high fixed-yield, multi-level referral (MLM-shaped) structure: DAILY_STATIC_BPS = 100 pays 1%/day of principal, capped at MAX_STATIC_BPS = 25000 (250% of principal); on top of that a ten-level referral ladder (LEVEL_DIFF_BPS 1%..10%) plus a peer stream (PEER_REWARD_BPS = 10% of the level-9 reward) is accrued to uplines. All of these are paid out of a reward vault that must be actively replenished (fund_reward_vault; InsufficientVault error on empty). Obligations therefore grow with participation while funding is discretionary, which is the classic sustainability risk of yield/referral programs: if inflows or funding stall, later claimants cannot be paid even though the accounting says they are owed.

Recommendation

Document the funding model and reward liabilities explicitly for participants; consider tying reward emission to actual vault solvency (cap accrual to funded balance) rather than accruing unfunded obligations; disclose that returns are not guaranteed and depend on continued funding. This is a design/economic risk rather than a code defect.

Mitigation

Open - inherent to the reward parameters; requires product/economic decision.

References:

N/A

LOOP-03 | Raydium Pool / AMM-Config State Parsed By Hardcoded Byte Offsets.

Category	Severity	CVSS	Location	Status
External Dependencies	 Medium	5.3	src/*.rs: utils.rs: read_pool_fee_state, r ead_amm_config_fee_ rates_from_bytes, get_amount_in	 Attention Required

Description

buy_a8 pricing reads the Raydium CPMM pool and amm_config accounts by fixed byte offsets (pool_state: 72/104 vaults, 341/349/357/365 protocol+fund fees, 389/390 creator-fee flags, 397/405 creator fees; amm_config: 12 trade_fee_rate, 108 creator_fee_rate) and re-implements Raydium's constant-product-with-fees math off-chain (get_amount_in, raydium_pre_fee_amount, vault_amount_without_fee). Two risks: (1) if Raydium changes its account layout or fee semantics, these offset reads silently return wrong values, mispricing the SOL->A8 buy (over/undercharge, or SlippageExceeded DoS); (2) read_pool_fee_state performs no account-discriminator check and does not, in the reviewed code, confirm the passed pool/amm_config are the configured a8_pool - that identity check must be enforced in the (un-reviewed) handler. A buffer_bps is applied to absorb estimation error, which mitigates but does not remove the layout-fragility risk.

Recommendation

Prefer Raydium's own quote/CPI or a versioned deserializer over raw offsets; pin and assert the expected account discriminator/owner; and, in the handler, require the pool/amm_config/vault keys to equal the values stored in StakeConfig (a8_pool, a8_pool_a8_vault, a8_pool_sol_vault). Add a test that fails if the Raydium layout version changes.

Mitigation

Open - offset parsing present in reviewed code; caller-side identity checks not reviewed (out of scope).

References:

N/A

LOOP-04 | Token Amount Read From Raw Account Bytes Without Owner/Type Check.

Category	Severity	CVSS	Location	Status
Input Validation	 Medium	5.5	src/*.rs: utils.rs: read_token_amount	 Attention Required

Description

read_token_amount interprets bytes 64..72 of an arbitrary AccountInfo as an SPL token balance with no verification that the account is owned by the SPL Token (or Token-2022) program and no mint check. It is used to read the pool SOL/A8 vault reserves that feed swap pricing. If a handler passes an unconstrained or attacker-controlled account into this helper, the reserve figures - and therefore the computed SOL cost of a buy - can be forged. Whether every call site constrains these accounts (e.g. Account<TokenAccount> with an address/has_one constraint) is decided in the instruction-handler layer, which was not provided; at the reviewed-code level the helper itself is unsafe by construction.

Recommendation

Deserialize via anchor_spl TokenAccount (which enforces the token-program owner) or explicitly assert `account.owner == spl_token::ID / token_2022::ID` and validate the mint before trusting the amount. Constrain the vault accounts to the pool-derived keys at the `#[derive(Accounts)]` level.

Mitigation

Open - final impact depends on handler account constraints (not provided / out of scope).

References:

N/A

LOOP-05 | Referral Upline Chain Is Caller-Supplied; Completeness/Length Enforced Outside Reviewed Code.

Category	Severity	CVSS	Location	Status
Access Control	 Medium	5.5	src/*.rs: utils.rs: settle_ dynamic_rewards	 Attention Required

Description

settle_dynamic_rewards distributes level/peer rewards and maintains valid_team_count by iterating remaining_accounts, treating each as an upline StakeUser. Each entry is checked for program ownership and PDA validity (verify_stake_user_pda), and referral_settled prevents double-settlement per order - both good. However, the ORDER and COMPLETENESS of the upline ancestry are supplied by the caller: the reviewed function trusts that the passed accounts are the true, in-order upline chain. A caller passing a truncated, reordered or padded chain could distort reward payout and team-count bookkeeping. The loop is also unbounded in the reviewed code (no MAX_LEVEL cap here), so an over-long remaining_accounts list is a compute-budget/DoS concern. The IncompleteUpline error implies a completeness check exists, but that check is in the (un-reviewed) handler.

Recommendation

Enforce the upline chain deterministically: reconstruct/verify each upline from the stored referrer link rather than trusting caller order, cap the iteration to MAX_LEVEL, and confirm the handler rejects incomplete/oversized chains (IncompleteUpline) before any accrual. Add tests for truncated/reordered chains.

Mitigation

Open - chain-completeness enforcement is in the handler layer (out of scope); reviewed function trusts caller-supplied ordering.

References:

N/A

LOOP-06 | Admin-Settable seconds_per_day and Fixed UTC+8 Day Boundary Affect Accrual Timing.

Category	Severity	CVSS	Location	Status
Centralization	 Low	3.1	src/*.rs: utils.rs: day_index, day_start_ts / constants.rs: BEIJING_ OFFSET_SECONDS; lib.rs: set_seconds_per_day	 Attention Required

Description

Reward accrual and order maturity are measured in whole 'days' via `day_index`, which applies a hardcoded `BEIJING_OFFSET_SECONDS` (28800 = UTC+8) and divides by `seconds_per_day`. `seconds_per_day` is an admin-settable value (`set_seconds_per_day`). An admin lowering it effectively shortens a 'day', accelerating 1%/day static accrual and bringing maturity forward; raising it does the reverse. `day_start_ts` also computes `day_idx * seconds_per_day - offset` without a checked multiply. This ties reward timing to a mutable privileged parameter and a fixed timezone assumption.

Recommendation

Treat `seconds_per_day` as immutable after initialize (or constrain it to a narrow sane range and timelock changes); document the UTC+8 accrual boundary; use checked arithmetic in `day_start_ts`. Emit an event on any change.

Mitigation

Open - relates to the centralization surface (LOOP-01); timing parameter is mutable by design.

References:

N/A

LOOP-07 | Field maturity_day Stores A Unix Timestamp, Not A Day Index.

Category	Severity	CVSS	Location	Status
Code Quality	 Informational	0.0	src/*.rs: state.rs: Order.maturity_day; utils.rs	 Informational

Description

Despite the name, Order.maturity_day holds a unix timestamp (order_maturity_ts / day_start_ts return timestamps, and the tests set maturity_day = day_start_ts(...)). Consumers must remember to convert: static_settle_end_day calls day_index(order.maturity_day, ...) to get a day index, while is_order_counting_active compares now_ts < order.maturity_day as timestamps. The dual meaning of '..._day' fields (some are day indices, e.g. last_settle_day; some are timestamps, e.g. maturity_day/start_day) is error-prone for future maintenance.

Recommendation

Rename to maturity_ts / start_ts (or store true day indices consistently) and document the unit on each time field. No functional change required.

Mitigation

Informational - naming/maintainability only; reviewed logic is internally consistent.

References:

N/A

LOOP-08 | Release overflow-checks Not Verifiable; Confirm Enabled.

Category	Severity	CVSS	Location	Status
Best Practices	 Informational	0.0	src/*.rs: Cargo.toml (not provided)	 Informational

Description

The reviewed math uses explicit checked_* operations and u128 widening in the important paths (mul_bps, settle_order_static, get_amount_in, apply_buffer_bps), which is good. A few non-critical operations are unchecked (e.g. day_start_ts multiply). Because Cargo.toml was not provided, it cannot be confirmed that overflow-checks = true is set for the release profile - by default Solana release builds do NOT panic on overflow, so any unchecked arithmetic would wrap silently.

Recommendation

Set overflow-checks = true under [profile.release] in Cargo.toml and provide it for verification; keep using checked_* for value-bearing arithmetic regardless.

Mitigation

Informational - could not be verified from the provided files; recommend confirming.

References:

N/A

LOOP-09 | Orphan set_level Field Raises Reward Level With No Setter In The Reviewed Router.

Category	Severity	CVSS	Location	Status
Access Control	 Informational	0.0	src/*.rs: state.rs: StakeUser.set_level; utils.rs: reward_level	 Informational

Description

reward_level(user) returns user.level.max(user.set_level), so a non-zero set_level can raise a user's effective referral level (and thus their reward tier) above what their real team size earns. StakeUser carries a set_level field and error.rs defines InvalidSetLevel ('must be 0..=10'), implying a privileged instruction sets it - but no such instruction is exposed in the reviewed router (lib.rs). Its setter therefore lives in the un-reviewed handler layer. This should be confirmed to be an intended, access-controlled admin/operator feature and not an unguarded privilege path.

Recommendation

Confirm which instruction writes set_level, that it is restricted to admin/operator with proper validation (InvalidSetLevel range check), and document it as a privileged capability. Surface set_level in the privileged-roles disclosure.

Mitigation

Informational - setter not present in reviewed scope; confirm gating in handler layer.

References:

N/A

FINDINGS

In this document, we present the findings and results of the smart contract security audit. The identified vulnerabilities, weaknesses, and potential risks are outlined, along with recommendations for mitigating these issues. It is crucial for the team to address these findings promptly to enhance the security and trustworthiness of the smart contract code.

Severity	Found	Pending	Resolved
 Critical	0	0	0
 High	1	1	0
 Medium	4	4	0
 Low	1	1	0
 Informational	3	3	0
Total	9	9	0

In a smart contract, a technical finding summary refers to a compilation of identified issues or vulnerabilities discovered during a security audit. These findings can range from coding errors and logical flaws to potential security risks. It is crucial for the project owner to thoroughly review each identified item and take necessary actions to resolve them. By carefully examining the technical finding summary, the project owner can gain insights into the weaknesses or potential threats present in the smart contract. They should prioritize addressing these issues promptly to mitigate any risks associated with the contract's security. Neglecting to address any identified item in the security audit can expose the smart contract to significant risks. Unresolved vulnerabilities can be exploited by malicious actors, potentially leading to financial losses, data breaches, or other detrimental consequences. To ensure the integrity and security of the smart contract, the project owner should engage in a comprehensive review process. This involves understanding the nature and severity of each identified item, consulting with experts if needed, and implementing appropriate fixes or enhancements. Regularly updating and maintaining the smart contract's codebase is also essential to address any emerging security concerns. By diligently reviewing and resolving all identified items in the technical finding summary, the project owner can significantly reduce the risks associated with the smart contract and enhance its overall security posture.

SOCIAL MEDIA CHECKS | LOOP Protocol.

Social Media	URL	Result
Website	https://x.com/loop_protocol_	Confirmed
Telegram		N/A
Twitter	https://x.com/loop_protocol_	Confirmed
Facebook		N/A
Reddit	N/A	N/A
Instagram	N/A	N/A
CoinGecko		N/A
Github	N/A	N/A
CMC		N/A
Email		Contact
Other		N/A

From a security assessment standpoint, inspecting a project's social media presence is essential. It enables the evaluation of the project's reputation, credibility, and trustworthiness within the community. By analyzing the content shared, engagement levels, and the response to any security-related incidents, one can assess the project's commitment to security practices and its ability to handle potential threats.

Social Media Information Notes:

Auditor Notes: Project X/Twitter (x.com/loop_protocol_) provided by the client.

Project Owner Notes: Partial-scope engagement.

Assessment Results

Final Audit Score LOOP.

Review	Score
Reviewed scope	Router + state / constants / utils / raydium / error
Excluded (not provided)	instructions/*.rs handlers & account-validation
Security score (reviewed scope)	82 / 100

This is a partial source review. The program's core modules that were provided - the entry-point router (lib.rs), account/state layout (state.rs), constants (constants.rs), reward/settlement and Raydium-pool math (utils.rs), the Raydium CPI wrappers (raydium.rs) and error codes (error.rs) - were manually reviewed and are what the score below reflects. The instruction-handler layer (instructions/*.rs) - which contains every Anchor #[derive(Accounts)] context and therefore the program's signer, owner, has_one, PDA-seed and account-type validation - was NOT provided and is EXCLUDED from this review. The score below applies ONLY to the reviewed modules; because account validation is the primary Solana attack surface and was not seen, NO whole-program pass or fail is issued and no assurance is given for LOOP Protocol as a whole until the handler modules are reviewed. Passing benchmark for reference is 80 of 100.

PARTIAL REVIEW - REVIEWED SCOPE ONLY

Score 82/100 applies only to the reviewed modules (router, state, constants, utils, raydium, error). The instruction-handler / account-validation layer was not provided; no whole-program pass or fail is issued.

Important Notes for LOOP

Scope and Status

This report is a PARTIAL SOURCE REVIEW of LOOP Protocol, a Solana program built with the Anchor framework. It is SCORED, but the score reflects the reviewed scope only.

Reviewed modules: lib.rs (program entry-point / instruction router), state.rs (accounts and events), constants.rs (economic parameters), utils.rs (reward settlement, referral distribution and Raydium-pool math), raydium.rs (Raydium CPMM CPI wrappers) and error.rs (error codes).

Excluded from this review: the instruction-handler modules under instructions/ (initialize, admin, operator, stake, buy_a8, register, set_a9_price, claim_static, claim_principal, claim_dynamic, claim_peer, claim_all). These files were not provided. They contain every Anchor account-validation context, that is the signer, owner, has_one, PDA-seed, account-type and duplicate-account checks, together with the handler orchestration. Because account validation is the primary attack surface of a Solana program, the score below is NOT a whole-program pass or fail and no assurance is given for LOOP Protocol as a whole until those modules are reviewed.

What the reviewed code does well

The reviewed modules show sound engineering in several places. Value-bearing arithmetic uses checked operations and 128-bit widening. The Raydium CPI wrappers pin the callee to the real Raydium CPMM program id before every invocation. Upline accounts in the referral cascade are validated as program-owned StakeUser PDAs via create_program_address. Zero-copy account layouts carry compile-time size assertions. Per-order flags guard against double settlement of static rewards, principal and referral.

Key risks in the reviewed scope

Centralization is the headline item: prices, fund-routing wallets, staking parameters and the reward on/off switch are all privileged controls. The economic model is a high fixed-yield, multi-level referral structure funded from a vault that must be actively replenished, which carries the usual sustainability risk. Raydium pool and config state is parsed by hardcoded byte offsets and the fee math is re-implemented off-chain, which is fragile to any Raydium layout change. Two helper-level issues, a token amount read without an owner check and a caller-supplied referral chain, have a final impact that depends on the excluded handler constraints and should be confirmed there.

Next step

Provide the instructions/ handler modules, Cargo.toml and the deployed program address for a full-scope audit with a whole-program score and pass/fail status.

This assessment is not financial advice and not a guarantee. It reflects the state of the provided source at the time of review.

I Appendix

Finding Categories

Centralization / Privilege

Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Gas Optimization

Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction in the total gas cost of a transaction.

Logical Issue

Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion of how `block.timestamp` works.

Control Flow

Control Flow findings concern the access control imposed on functions, such as owner-only functions being invocable by anyone under certain circumstances.

Volatile Code

Volatile Code findings refer to segments of code that behave unexpectedly in certain edge cases that may result in a vulnerability.

Coding Style

Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Inconsistency

Inconsistency findings refer to functions that should seemingly behave similarly yet contain different code, such as a constructor assignment imposing different require statements on the input variables than a setter function.

Coding Best Practices

ERC 20 Coding Standards are a set of rules that each developer should follow to ensure the code meets a set of criteria and is readable by all developers.

Disclaimer

Scope. This report documents the results of a security assessment and smart contract audit performed by Bladepool / CFG NINJA (the "Auditor") for the project specified in this report. The assessment covers only the deliverables and code expressly listed in the engagement; any changes, integrations, or subsequent deployments are outside the scope.

No Warranty. The Auditor performs the assessment using industry-standard practices but makes no representations or warranties, express or implied, including any warranty of merchantability, fitness for a particular purpose, or non-infringement. The Auditor does not guarantee that the audited code is free of bugs, vulnerabilities, or exploitable issues.

Limitation of Liability. To the maximum extent permitted by law, the Auditor and its affiliates, officers, employees, agents, and contractors shall not be liable for any indirect, incidental, special, consequential, or punitive damages, or for any loss of profits, revenue, data, or business opportunities, arising out of or related to the assessment, even if advised of the possibility of such damages. The Auditor's aggregate liability for direct damages is limited to the fees paid for the audit engagement.

Client Responsibility. The project owner/client retains sole responsibility for all decisions and actions taken in connection with the audited code, including fixing identified issues, deploying code to any environment, and applying security mitigations. The Auditor's recommendations are advisory; implementation and verification of fixes are the client's responsibility.

Third-Party Tools and Services. The Auditor may use third-party tools, services, or automated scanners as part of the assessment. Use of such tools is without warranty; the Auditor is not responsible for defects, failures, or inaccuracies arising from third-party services.

Confidentiality and Data Security. The Auditor will treat non-public client information as confidential. However, the Auditor cannot guarantee absolute security for data transmitted over the internet or third-party platforms. Client should take reasonable precautions to protect sensitive information.

Not Financial, Legal, or Investment Advice. The assessment is a technical security review only and is not financial, legal, tax, or investment advice. Clients should consult appropriate professionals for those matters.

Ownership and Governance Notes. If ownership is transferred, held by a multisig, or controlled by a governance contract, the client should document and disclose those arrangements; the Auditor's report reflects the ownership state observed during the engagement but may not reflect subsequent changes.

Governing Law and Counsel. This engagement and any disputes arising from it shall be governed by the laws agreed in the engagement terms. Clients are advised to seek legal counsel before relying on or publishing the report.

Acceptance. By proceeding with this engagement or using this report, the client acknowledges and accepts these terms.

